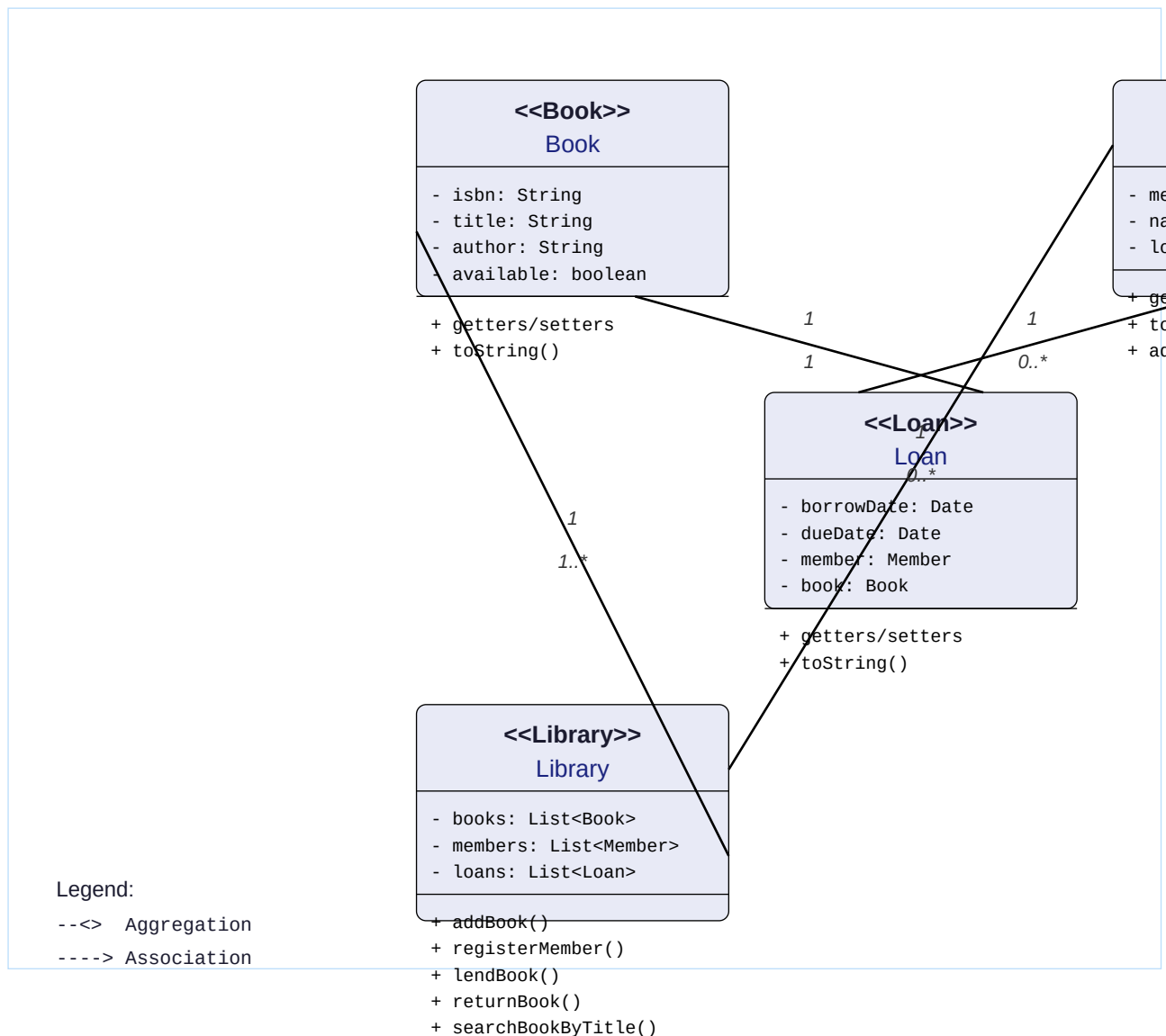


UML Class Diagram

Library Management System

Based on Java Implementation Reference

The diagram below shows the four classes and their relationships:



1. Class Descriptions

Book

Attributes:

- - isbn : String (private)
- - title : String (private)
- - author : String (private)
- - available : boolean (private)

Methods:

- + Book(isbn, title) (public)
- + Book(isbn, title, author) (public)
- + getIsbn() : String (public)
- + getTitle() : String (public)
- + getAuthor() : String (public)
- + getStatus() : boolean (public)
- + setStatus(available : boolean) : void (public)
- + toString() : String (public)

Member

Attributes:

- - memberId : String (private)
- - name : String (private)
- - loans : List (private)

Methods:

- + Member(memberId, name) (public)
- + getMemberId() : String (public)
- + getName() : String (public)
- + addLoan(loan : Loan) : void (public)
- + toString() : String (public)

Loan

Attributes:

- - borrowDate : Date (private)
- - dueDate : Date (private)
- - member : Member (private)
- - book : Book (private)

Methods:

- + Loan(member, book, bDate, dDate) (public)
- + getBorrowDate() : Date (public)
- + getDueDate() : Date (public)

- + getMember() : Member (public)
- + getBook() : Book (public)
- + toString() : String (public)

Library

Attributes:

- - books : List (private)
- - members : List (private)

Methods:

- + addBook(book : Book) : void (public)
- + registerMember(member : Member) : void (public)
- + lendBook(borrower, book, bDate, dDate) : void (public)
- + returnBook(book : Book) : void (public)
- + searchBookByTitle(title : String) : Book (public)

2. Visibility Symbols

In UML, visibility symbols indicate the accessibility of class members:

Symbol	Visibility	Meaning
+	Public	Accessible from any other class.
-	Private	Accessible only within the class itself.
#	Protected	Accessible within the class and subclasses.
~	Package	Accessible within the same package.

In our system, all attributes are **private (-)** and all methods are **public (+)**, following proper encapsulation principles.

3. Relationships

3.1 Association

Definition: Association represents a structural relationship between two or more classes, indicating that objects of one class are connected to objects of another class. It is the most basic form of relationship in OOP and UML.

Examples in our system:

- **Member ↔ Loan:** A Member borrows books, and each borrowing is recorded as a Loan. The Member class holds a list of Loans (*loans : List<Loan>*), and the Loan class holds a reference to the Member (*member : Member*). This is a bi-directional association.
- **Loan ↔ Book:** A Loan is associated with exactly one Book. The Loan class holds a reference *book : Book* to indicate which book was borrowed.

Key points about Association:

- It can be unidirectional (one class knows about the other) or bidirectional (both classes know about each other).
- It can have a label (e.g., "borrows") and multiplicity at each end.
- In Java, associations are typically implemented as reference fields (object pointers).

3.2 Aggregation

Definition: Aggregation is a specialized form of Association that represents a "has-a" or "whole-part" relationship where the part can exist independently of the whole. It is often called "weak aggregation" because the lifecycle of the part is not tied to the lifecycle of the whole.

Why Library and Book represent Aggregation:

A Library *has* Books, but a Book can exist independently of the Library. A Book could be removed from this Library and added to another Library, or it could exist before being added to any Library. The Book has its own independent lifecycle. This is the defining characteristic of aggregation — the "part" (Book) can exist without the "whole" (Library).

Why Library and Member represent Aggregation:

Similarly, a Library *has* Members, but a Member is a person who can exist independently. A Member could be registered in multiple libraries or move between libraries. The Member object has its own lifecycle separate from the Library.

UML Notation: Aggregation is shown as a hollow diamond on the "whole" side (Library) connected by a line to the "part" side (Book or Member).

3.3 Composition

Definition: Composition is a stronger form of Aggregation where the part cannot exist independently of the whole. If the whole is destroyed, the parts are also destroyed. This is a "strong has-a" or "owns-a" relationship with lifecycle dependency.

How Composition differs from Aggregation:

Feature	Aggregation	Composition
Lifecycle	Parts exist independently	Parts depend on the whole

Relationship	"Has-a" (weak)	"Owns-a" (strong)
UML Notation	Hollow diamond	Filled diamond
Example	Library <> Book	House <> Room

Practical Example — House and Room:

A House contains Rooms. If the House is demolished (destroyed), the Rooms cease to exist as well. A Room cannot exist without being part of some House. This is Composition. In contrast, a Library contains Books, but if the Library closes, the Books still exist and can be transferred elsewhere — this is Aggregation.

4. Multiplicity

Definition: Multiplicity defines how many instances of one class can be associated with instances of another class in a relationship. It specifies the quantitative aspect of the relationship.

Common multiplicity values:

Notation	Meaning	Example
1	Exactly one instance	Loan -- 1 -- Book (one loan is for exactly one book)
0..1	Zero or one instance (optional)	A person may have zero or one passport
0..*	Zero or more instances (any number)	Member -- 0..* -- Loan (a member can have zero or many loans)
1..*	One or more instances (at least one)	Library -- 1..* -- Book (a library must have at least one book)

Multiplicity examples from our system:

- **Member 1 — 0..* Loan:** One Member can have zero or more Loans. A new member may have no loans yet, but over time they can borrow many books.
- **Loan 1 — 1 Book:** Each Loan is for exactly one Book. A loan cannot be for multiple books.
- **Library 1 — 1..* Book:** A Library must contain at least one Book but can hold many. It would not be a library with zero books.
- **Library 1 — 0..* Member:** A Library can have zero or more registered Members. A new library may start with no members and grow over time.

5. Summary

The UML Class Diagram for the Library Management System models four key classes (**Library**, **Book**, **Member**, **Loan**) with the following relationships:

- **Aggregation** between Library and Book (hollow diamond) — Library has Books, but Books exist independently.
- **Aggregation** between Library and Member (hollow diamond) — Library registers Members, but Members exist independently.
- **Association** between Member and Loan — Members take out Loans.
- **Association** between Loan and Book — A Loan references a Book.

All attributes are **private (-)** to enforce encapsulation, and methods are **public (+)** to provide controlled access to the object's state. Multiplicities define the cardinality of each relationship, ensuring the model accurately reflects real-world library operations.